

Introduction to OpenMP

Lecture 3: Parallel Regions

Parallel region directive

- Code within a parallel region is executed by all threads.
- Syntax:

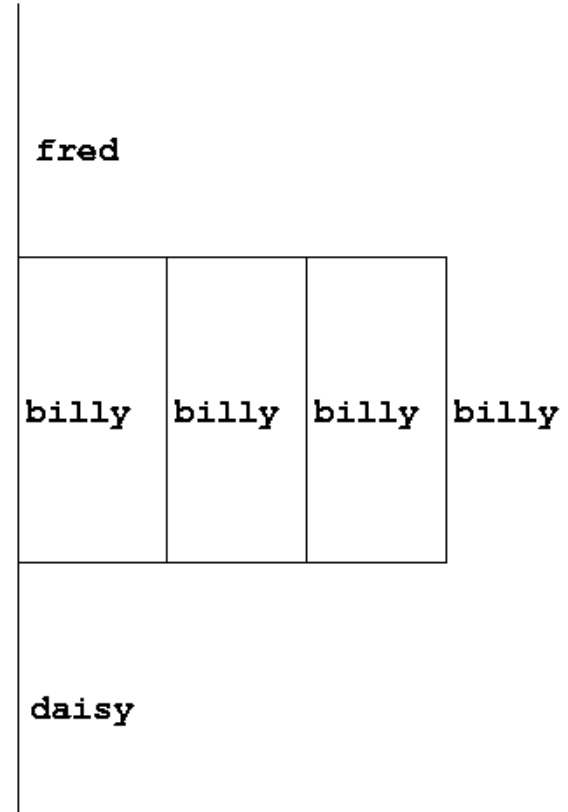
```
C/C++:  #pragma omp parallel
        {
            code block
        }
```

The code block must be *structured*: jumps in or out of it are not allowed.
(this is the case for most OpenMP constructs).

Parallel region directive (cont)

Example:

```
fred();  
#pragma omp parallel  
{  
    billy();  
}  
daisy();
```



Useful functions

- Often useful to find out number of threads being used.

C/C++:

```
#include <omp.h>

int omp_get_num_threads(void);
```

- **Important note:** returns 1 if called outside parallel region!

Useful functions (cont)

- Also useful to find out number of the executing thread.

C/C++:

```
#include <omp.h>
int omp_get_thread_num(void)
```

- Takes values between 0 and `omp_get_num_threads() - 1`

Clauses

- Specify additional information in the parallel region directive through clauses:
- C/C++: `#pragma omp parallel [clauses]`
- Clauses are comma or space separated.

Shared and private variables

- Inside a parallel region, variables can be either **shared** (all threads see same copy) or **private** (each thread has its own copy).
- Can specify the behaviour of variables explicitly using **shared** and **private** clauses:

shared (*list*)

private (*list*)

where *list* is a comma-separated list of variables

- Can also specify the default behaviour for variables not explicitly listed:

default (**shared|none**)

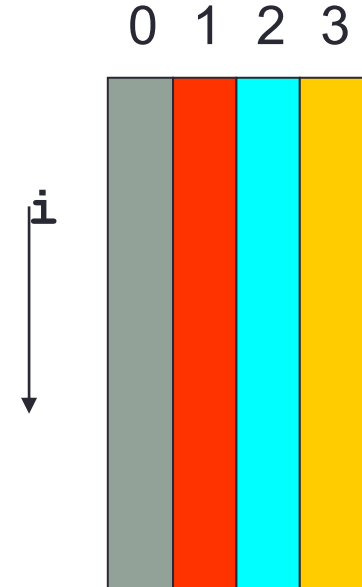
Shared and private (cont.)

- On entry to a parallel region, private variables are *uninitialised*.
- Variables declared inside the scope of the parallel region are automatically private.
- After the parallel region ends the original variable is unaffected by any changes to private copies.
- In C++ private objects are created using the default constructor.
- Not specifying a **default** clause is the same as specifying **default(shared)**
- **Danger!** - always use **default(none)**

Shared and private (cont)

Example: each thread initialises its own part of a shared array:

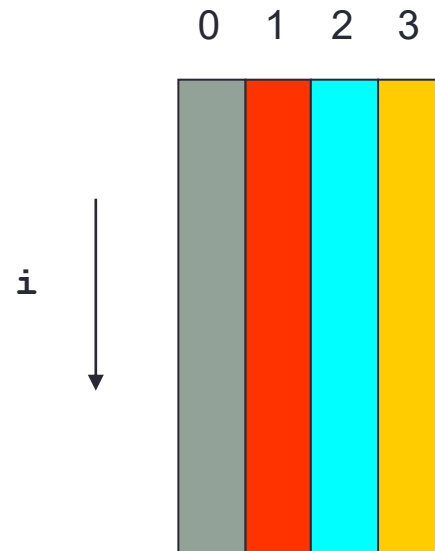
```
int i,myid,n=1000;  
float a[4][1000];  
  
#pragma omp parallel default(none) private(i,myid) shared(a,n)  
{  
    myid = omp_get_thread_num();  
    for (i=0; i<n; i++){  
        a[myid][i] = 1.0;  
    }  
}
```



Shared and private (cont)

Using the C language scoping:

```
int n=1000;  
float a[4][1000];  
  
#pragma omp parallel default(none) shared(a,n)  
{  
    int myid = omp_get_thread_num();  
    for (int i=0; i<n; i++){  
        a[myid][i] = 1.0;  
    }  
}
```



10

Multi-line directives

- C/C++:

```
#pragma omp parallel default(none) \  
private(i,myid) shared(a,n)
```

Note: no white space after the \

Initialising private variables

- Private variables are uninitialised at the start of the parallel region.
- If we wish to initialise them, we use the **firstprivate** clause instead of **private**:

C/C++: **firstprivate** (*list*)

- Private copies are initialised with the value in the original variable at the start of the parallel region
- Note: use cases for this are uncommon!
- In C++ the default copy constructor is called to create and initialise the new object

Initialising private variables (cont)

Example:

```
    int n = 1000;
    float b = 23.0;
    float c[4][1000];
    . . . . .
#pragma omp parallel default(none) firstprivate(b) \
shared(c,n)
{
    int myid = omp_get_thread_num();
    for (int i=0; i<n-1; i++){
        b += c[myid][i];
    }
    c[myid][n-1] = b;
}
```

Reductions

- A *reduction* produces a single value from associative operations such as addition, multiplication, max, min, and, or.
- Each thread will reduce into a private copy, then combine all of these to give final result.
- Use **reduction** clause:

reduction (op: list)

- **op** can be one of: **+**, *****, **&**, **|**, **^**, **&&**, **||**, **max**, **min**
- variables must be of basic type
- initial value of private copies depends on the operator (e.g. 0 for **+**, 1 for *****, most negative value in type for **max**).

Reductions (cont.)

Example:

```
b = 10;
#pragma omp parallel reduction(+:b)
{
    int myid = omp_get_thread_num();
    for (int i=0; i<n; i++){
        b += c[myid][i];
    }
}
a = b;
```

Value in original variable is saved

Each thread gets a private copy of **b**, initialised to 0

All accesses inside the parallel region are to the private copies

At the end of the parallel region, all the private copies are added into the original variable

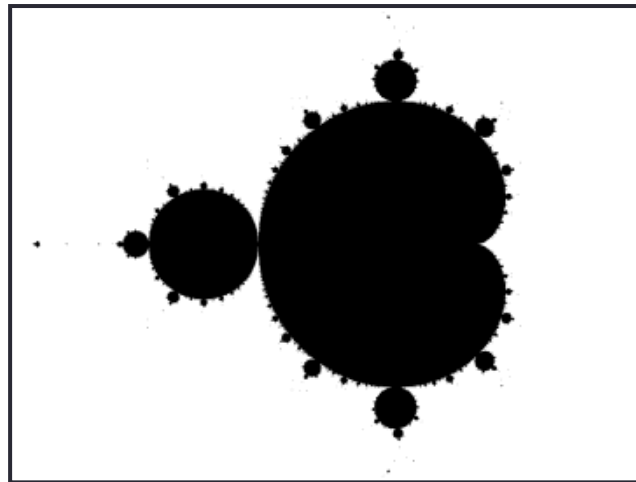
Arrays

- Can have private, firstprivate or reduction arrays
- In C/C++, need to use a special OpenMP syntax for array sections:
`array[firstelement:numofelements]`
 - necessary since the compiler cannot determine the length of dynamically allocated storage
- Reductions are done on each element of arrays separately
 - result is an array of the same size (not a scalar!)

Exercise

Area of the Mandelbrot set

- Aim: introduction to using parallel regions.
- Estimate the area of the Mandelbrot set.
 - Generate a grid of complex numbers in a box surrounding the set
 - Test each number to see if it is in the set or not.
 - Ratio of points inside to total number of points gives an estimate of the area.
 - Testing of points is independent - parallelise with a parallel region!



Fortran syntax

Parallel region syntax:

```
!$OMP PARALLEL [clauses]  
    block  
!$OMP END PARALLEL
```

Useful functions:

```
USE OMP_LIB
```

```
INTEGER FUNCTION OMP_GET_NUM_THREADS()
```

```
INTEGER FUNCTION OMP_GET_THREAD_NUM()
```

Fortran syntax

Data clauses:

SHARED (*list*)

PRIVATE (*list*)

FIRSTPRIVATE (*list*)

REDUCTION (*op*:*list*)

DEFAULT (SHARED|PRIVATE|NONE)

- *op* can be one of: +, *, .and., .or., .eqv., .neqv., max, min, iand, ior, ieor
- No special syntax need for arrays (but can use normal Fortran subarrays).

Fortran syntax

Multiline directives: fixed source form

```
!$OMP PARALLEL DEFAULT (NONE) , PRIVATE (I,MYID) ,  
!$OMP&  SHARED (A,N)
```

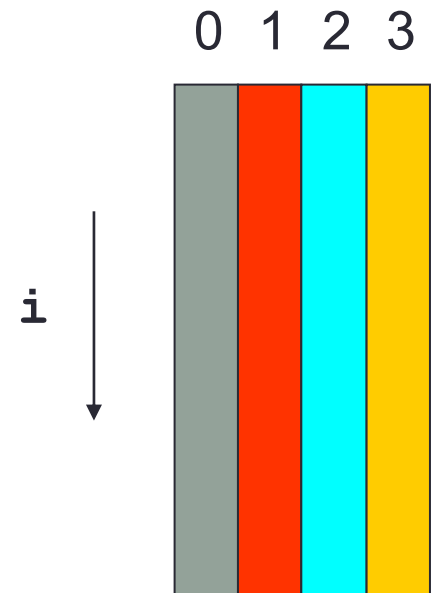
Multiline directives: free source form

```
!$OMP PARALLEL DEFAULT (NONE) , PRIVATE (I,MYID) , &  
!$OMP SHARED (A,N)
```

Fortran example

Example: each thread initialises its own part of a shared array:

```
!$OMP PARALLEL DEFAULT(NONE) , PRIVATE(I,MYID) , SHARED(A,N)
  myid = omp_get_thread_num() + 1
  do i = 1,n
    a(i,myid) = 1.0
  end do
!$OMP END PARALLEL
```



Fortran example - firstprivate

Example:

```
b = 23.0
```

```
. . . . .
```

```
!$OMP PARALLEL FIRSTPRIVATE(B) PRIVATE(I,MYID)
```

```
myid = omp_get_thread_num() + 1
```

```
do i = 1,n-1
```

```
    b = b + c(i,myid)
```

```
end do
```

```
c(n,myid) = b
```

```
!$OMP END PARALLEL
```

Fortran example - reduction

Example:

```
b = 10
!$OMP PARALLEL REDUCTION(+:b) ,
!$OMP& PRIVATE(I,MYID)
  myid = omp_get_thread_num() + 1
  do i = 1,n
    b = b + c(i,myid)
  end do
!$OMP END PARALLEL
a = b
```

Value in original variable is saved

Each thread gets a private copy of **b**, initialised to 0

All accesses inside the parallel region are to the private copies

At the end of the parallel region, all the private copies are added into the original variable